60. LLD (Low-Level Design): Complete Learning Guide

Uses ideas you already know from **#04 (controller vs service)**, **#06 (decorators)**, **#07 (dependency injection)** and **#12-14 (interface, DTO)**.

> **In one line:** LLD is **how you design the code inside one app**: which **classes**, **interfaces** and **methods** exist, how they **talk to each other**, and how to keep the code **easy to change** later.

HLD vs LLD (quick reminder)

	HLD (High-Level)	LLD (Low-Level)
Question	"Which servers, DBs, queues?"	"Which classes, interfaces, methods?"
Real life	City plan: roads, power plants	One house: rooms, doors, wiring
Output	Architecture diagram	Class diagram + code
Example	Design WhatsApp for 1 billion users	Design a Parking Lot system in code

The roadmap: what to learn, in order

```
STEP 1  OOP basics            class, object, 4 pillars          (3-4 days)
   ↓
STEP 2  Class relationships   association, aggregation,         (2 days)
                              composition, inheritance
   ↓
STEP 3  SOLID principles      5 rules for clean code            (1 week)
   ↓
STEP 4  UML class diagrams    drawing your design               (2 days)
   ↓
STEP 5  Design patterns       8 must-know patterns              (2 weeks)
   ↓
STEP 6  Extra principles      DRY, KISS, YAGNI, composition     (1 day)
                              over inheritance
   ↓
STEP 7  LLD problems          Parking Lot, Library, BookMyShow  (2-3 weeks)
                              (practise the interview steps)
```

STEP 1: OOP basics

1. Class and object

> **Class** = the **blueprint** of a car (design on paper). **Object** = a **real car** made from that blueprint. One blueprint, many cars.

```
class Student {
  constructor(public name: string, private marks: number) {}
  isPass(): boolean { return this.marks >= 35; }
}

const s1 = new Student('Ravi', 80);   // object 1
const s2 = new Student('Asha', 20);   // object 2
```

2. The 4 pillars of OOP

Pillar	Meaning	Real life
Encapsulation	Keep data **private** and change it only through methods	An ATM: you can't touch the cash, only use the buttons
Abstraction	Show **what** it does and hide **how**	Car pedal: press it to go, no need to know the engine
Inheritance	A child class **reuses** the parent's code	Child gets features from the parent
Polymorphism	The **same method name** does different work	`pay()` by card, UPI or cash

```
// Encapsulation
class BankAccount {
  private balance = 0;                          // hidden
  deposit(amount: number) {
    if (amount <= 0) throw new Error('Invalid');
    this.balance += amount;
  }
  getBalance() { return this.balance; }
}

// Abstraction + Polymorphism
interface Payment { pay(amount: number): void; }          // WHAT
class UpiPayment  implements Payment { pay(a: number) { console.log(`UPI ${a}`); } }
class CardPayment implements Payment { pay(a: number) { console.log(`Card ${a}`); } }

function checkout(p: Payment) { p.pay(500); }   // works with ANY payment
```

```
checkout(new UpiPayment());
checkout(new CardPayment());

// Inheritance
abstract class Vehicle {
  constructor(public number: string) {}
  abstract wheels(): number;
}
class Car  extends Vehicle { wheels() { return 4; } }
class Bike extends Vehicle { wheels() { return 2; } }
```

STEP 2: Class relationships

3. How classes are connected

Relationship	Meaning	Real life	Strength
Inheritance (IS-A)	Child **is a** type of parent	Car **is a** Vehicle	Strongest
Composition (HAS-A, strong)	The part **can't live** without the whole	House has Rooms. No house, no rooms.	Strong
Aggregation (HAS-A, weak)	The part **can live** alone	Team has Players. A player exists without the team.	Medium
Association (USES)	They just **know** each other	Teacher teaches Student	Weak

```
// Composition: Order creates and owns its items
class Order {
  private items: OrderItem[] = [];
  addItem(name: string, qty: number) { this.items.push(new OrderItem(name, qty)); }
}

// Aggregation: Team receives players created outside
class Team {
  constructor(private players: Player[]) {}
}

// Association: a method just uses another object
class Teacher {
  grade(student: Student) { /* ... */ }
}
```

STEP 3: SOLID principles (most important)

4. S: Single Responsibility

One class = one job. A chef cooks, a waiter serves. If the chef also handles billing, the kitchen becomes a mess.

```
// BAD: one class does everything
class StudentService {
  create() { /* save to DB */ }
  sendEmail() { /* SMTP */ }
  generatePdf() { /* PDF */ }
}

// GOOD: each class has one job (this is why NestJS splits controller / service, #04)
class StudentService { create() {} }
class MailService    { sendEmail() {} }
class PdfService     { generatePdf() {} }
```

5. O: Open/Closed

Open for adding, closed for changing. Add a new feature with a new class, without editing old working code. It's like a phone: you install new apps, you don't rewrite the operating system.

```
// BAD: every new discount means editing this function
function discount(type: string, price: number) {
  if (type === 'student') return price * 0.9;
  if (type === 'festival') return price * 0.8;
  // add another if... and risk breaking the old ones
}

// GOOD: a new discount means a new class, and old code is untouched
interface Discount { apply(price: number): number; }
class StudentDiscount  implements Discount { apply(p: number) { return p * 0.9; } }
class FestivalDiscount implements Discount { apply(p: number) { return p * 0.8; } }
class DiwaliDiscount   implements Discount { apply(p: number) { return p * 0.7; } }  // new
```

6. L: Liskov Substitution

A child must work anywhere the parent works, with no surprises. If "Bird" can `fly()`, a Penguin child breaks that promise.

```
// BAD
class Bird { fly() {} }
class Penguin extends Bird { fly() { throw new Error('cannot fly'); } }  // surprise!

// GOOD: split the abilities
class Bird {}
interface FlyingBird { fly(): void; }
class Sparrow extends Bird implements FlyingBird { fly() {} }
class Penguin extends Bird { swim() {} }
```

7. I: Interface Segregation

> **Many small interfaces are better than one big one.** Don't force a class to implement methods it doesn't need. A TV remote doesn't need AC buttons.

```
// BAD: a basic printer is forced to "scan" and "fax"
interface Machine { print(): void; scan(): void; fax(): void; }

// GOOD
interface Printer { print(): void; }
interface Scanner { scan(): void; }
class BasicPrinter implements Printer { print() {} }
class OfficeMachine implements Printer, Scanner { print() {} scan() {} }
```

8. D: Dependency Inversion

> **Depend on an interface, not a concrete class.** A plug socket works with any device that has the right plug. It isn't wired to one fan. **NestJS dependency injection (#07) is built on this.**

```
// BAD: OrderService is welded to Gmail
class OrderService {
  private mailer = new GmailMailer();
}

// GOOD: depends on the interface, and NestJS injects the real class
export interface Mailer { send(to: string, msg: string): Promise<void>; }

@Injectable()
export class OrderService {
  constructor(@Inject('MAILER') private mailer: Mailer) {}
  async place() { await this.mailer.send('a@b.com', 'Order placed'); }
}

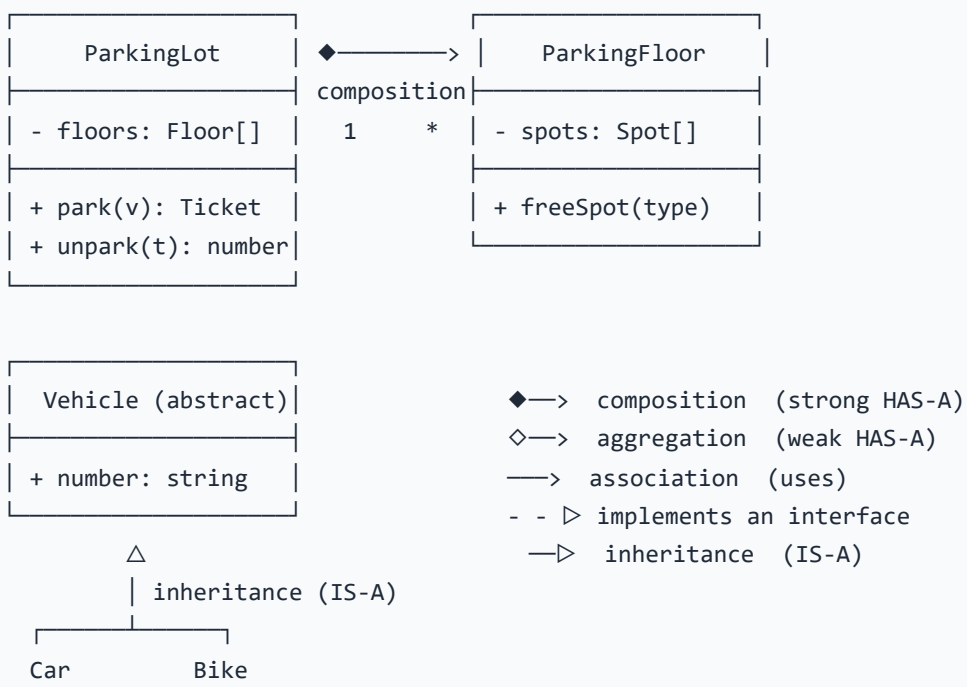
// app.module.ts: swap Gmail for SES by changing ONE line
providers: [{ provide: 'MAILER', useClass: SesMailer }]
```

Letter	Rule	One-word memory
S	Single Responsibility	One job
O	Open/Closed	Extend, don't edit
L	Liskov Substitution	Child = parent
I	Interface Segregation	Small interfaces
D	Dependency Inversion	Depend on an interface

STEP 4: UML class diagrams

9. Drawing your design

In interviews, you draw boxes for classes. Each box has 3 parts: **name**, **fields** and **methods**. `+` means public, `-` private, `#` protected.



STEP 5: Design patterns

10. What a design pattern is

A **ready-made solution to a common problem**, like a recipe. You don't invent a new way to make tea every time. There are 23 classic patterns (the "Gang of Four" book, 1994) in 3 groups. **Learn these 8 first.**

Group	Question it answers	Must-know
Creational	How are objects **created**?	Singleton, Factory, Builder
Structural	How are objects **combined**?	Adapter, Decorator, Facade
Behavioral	How do objects **talk / behave**?	Strategy, Observer

11. Singleton: only one object ever

> There's one Prime Minister, and everyone talks to the same one. Used for a DB connection, logger or config.

```
class Config {
  private static instance: Config;
  private constructor() {}                    // no "new Config()" from outside
  static get() {
    if (!this.instance) this.instance = new Config();
    return this.instance;
  }
}
Config.get() === Config.get();   // true
// NestJS: every @Injectable() service is a singleton by default
```

12. Factory: let one place create objects

> A pizza shop: you say "veg" or "chicken", and the kitchen decides how to make it. You never cook it yourself.

```
class NotificationFactory {
  static create(type: 'email' | 'sms' | 'push'): Notifier {
    switch (type) {
      case 'email': return new EmailNotifier();
      case 'sms':   return new SmsNotifier();
      case 'push':  return new PushNotifier();
```

```
    }
  }
}
NotificationFactory.create('sms').send('Fees due tomorrow');
// NestJS: useFactory in providers is this pattern
```

13. Builder: build big objects step by step

> A Subway sandwich: bread, then cheese, then veggies, then sauce. You choose step by step.

```
const query = new QueryBuilder()
  .select('name', 'marks')
  .from('students')
  .where('marks > 35')
  .limit(10)
  .build();
// TypeORM's createQueryBuilder() (#47) is the Builder pattern
```

14. Adapter: make incompatible things work together

> A travel plug adapter: an Indian plug in a UK socket.

```
// our app expects: pay(amountInRupees)
interface PaymentGateway { pay(rupees: number): void; }

// the 3rd-party Stripe SDK expects: charge(paise, currency)
class StripeAdapter implements PaymentGateway {
  constructor(private stripe: StripeSdk) {}
  pay(rupees: number) { this.stripe.charge(rupees * 100, 'INR'); }
}
```

15. Decorator: add features without changing the class

> Coffee + milk + sugar + cream. Each one wraps the coffee and adds something.

```
// NestJS uses this everywhere (#06): @UseGuards, @UseInterceptors
class LoggingRepo implements StudentRepo {
  constructor(private inner: StudentRepo) {}
  async findAll() {
    console.time('findAll');
    const r = await this.inner.findAll();   // original behaviour
    console.timeEnd('findAll');             // extra feature added
    return r;
```

```
  }
}
```

16. Facade: one simple door to a complex system

> A hotel reception: one desk for booking, food, cabs and laundry. You don't visit each department.

```
class CheckoutFacade {
  constructor(private stock: StockService, private pay: PaymentService,
              private ship: ShippingService, private mail: MailService) {}
  async placeOrder(order: Order) {          // one call for the controller
    await this.stock.reserve(order);
    await this.pay.charge(order);
    await this.ship.schedule(order);
    await this.mail.confirm(order);
  }
}
```

17. Strategy: swap the algorithm at runtime

> Google Maps route: by car, bike or walk. Same app, different way to calculate.

```
interface FareStrategy { fare(km: number): number; }
class NormalFare implements FareStrategy { fare(km: number) { return km * 10; } }
class SurgeFare  implements FareStrategy { fare(km: number) { return km * 18; } }

class Ride {
  constructor(private strategy: FareStrategy) {}
  setStrategy(s: FareStrategy) { this.strategy = s; }  // swap anytime
  total(km: number) { return this.strategy.fare(km); }
}
```

18. Observer: tell everyone when something happens

> A YouTube subscription: when a video is uploaded, every subscriber is notified.

```
class OrderEvents {
  private listeners: ((id: number) => void)[] = [];
  subscribe(fn: (id: number) => void) { this.listeners.push(fn); }
  emit(id: number) { this.listeners.forEach(fn => fn(id)); }
}

const events = new OrderEvents();
```

```
events.subscribe(id => console.log(`email for order ${id}`));
events.subscribe(id => console.log(`update stock for ${id}`));
events.emit(101);
// NestJS: @nestjs/event-emitter with @OnEvent('order.created'); also RxJS
```

Patterns you already used in NestJS

Pattern	Where in NestJS
Singleton	Every `@Injectable()` service (default scope)
Factory	`useFactory`, `TypeOrmModule.forRootAsync`
Builder	TypeORM `createQueryBuilder()`
Decorator	`@Controller`, `@UseGuards`, interceptors
Dependency Injection	Constructor injection everywhere
Chain of Responsibility	Middleware → guards → pipes → handler
Observer	EventEmitter, RxJS in interceptors
Repository	`Repository<Student>`

Learn later (after the 8)

Chain of Responsibility, State (order status), Command (undo/redo), Template Method, Proxy, Composite, Iterator, Prototype.

STEP 6: Extra principles

19. Small rules, big impact

Rule	Meaning
DRY: Don't Repeat Yourself	The same logic in 3 places means a bug fixed in only 1. Put it in one function.
KISS: Keep It Simple	Simple code beats clever code.
YAGNI: You Aren't Gonna Need It	Don't build features "just in case".
Composition over inheritance	Prefer "has-a" (inject parts) to deep `extends` chains.
Program to an interface	Depend on `Payment`, not on `UpiPayment`.

STEP 7: Solving LLD problems

20. Interview steps (use every time)

```
1. CLARIFY       ask questions: which vehicles? how many floors? payment?     (5 min)
2. REQUIREMENTS  write a short list of what it must do                        (5 min)
3. ENTITIES      find the nouns  -> classes  (ParkingLot, Floor, Spot, Ticket)
4. RELATIONS     has-a / is-a, draw the class diagram                         (10 min)
5. METHODS       find the verbs  -> methods  (park, unpark, calculateFee)
6. PATTERNS      where do Strategy / Factory / Singleton fit?
7. CODE          write the core classes                                       (20 min)
8. EXTEND        "how would you add EV charging?" -> show Open/Closed
```

21. Mini example: Parking Lot

Requirements: several floors, spot types (bike, car, truck), issue a ticket when a car enters, and calculate the fee by hours when it leaves.

```
enum VehicleType { BIKE, CAR, TRUCK }

abstract class Vehicle { constructor(public number: string, public type: VehicleType) {} }
class Car  extends Vehicle { constructor(n: string) { super(n, VehicleType.CAR); } }
class Bike extends Vehicle { constructor(n: string) { super(n, VehicleType.BIKE); } }
```

```
class Spot {
  vehicle?: Vehicle;
  constructor(public id: string, public type: VehicleType) {}
  isFree() { return !this.vehicle; }
}

class Floor {
  constructor(public no: number, private spots: Spot[]) {}
  freeSpot(type: VehicleType) { return this.spots.find(s => s.isFree() && s.type === type); }
}

class Ticket { constructor(public id: string, public spot: Spot, public entry = new Date()) {}
}

interface FeeStrategy { fee(hours: number, type: VehicleType): number; }   // Strategy
class HourlyFee implements FeeStrategy {
  private rate = { [VehicleType.BIKE]: 10, [VehicleType.CAR]: 30, [VehicleType.TRUCK]: 60 };
  fee(h: number, t: VehicleType) { return Math.max(1, Math.ceil(h)) * this.rate[t]; }
}

class ParkingLot {                                                  // Singleton in real app
  constructor(private floors: Floor[], private feeStrategy: FeeStrategy) {}

  park(v: Vehicle): Ticket {
    for (const f of this.floors) {
      const spot = f.freeSpot(v.type);
      if (spot) { spot.vehicle = v; return new Ticket(`T-${Date.now()}`, spot); }
    }
    throw new Error('Parking full');
  }

  unpark(t: Ticket): number {
    const hours = (Date.now() - t.entry.getTime()) / 3_600_000;
    const fee = this.feeStrategy.fee(hours, t.spot.vehicle!.type);
    t.spot.vehicle = undefined;
    return fee;
  }
}
```

> **What the interviewer checks:** clear classes (S), new vehicle types without editing `ParkingLot` (O), fee rules swappable (Strategy), and the design explained out loud.

22. Practice problems (easy to hard)

Level	Problem	Key patterns / ideas
Easy	Parking Lot	Inheritance, Strategy (fee)
Easy	Library Management	Entities, fines, search
Easy	Tic-Tac-Toe / Snake & Ladder	Game loop, Player, Board
Medium	Vending Machine	**State** pattern
Medium	ATM	State, Chain of Responsibility (notes)
Medium	Splitwise	Strategy (equal / exact / percent split)
Medium	Logger / Rate limiter	Singleton, Chain, Strategy
Hard	BookMyShow	Seat locking, concurrency, Observer
Hard	Elevator system	State, Strategy (scheduling)
Hard	Food delivery (Swiggy) / Uber	Strategy (matching), Observer (tracking)

23. 6-week study plan

Week	Learn	Practise
1	OOP pillars + class relationships	Model Student, Course, Teacher in TypeScript
2	SOLID (one per day)	Refactor your `student-pg` module using SOLID
3	Singleton, Factory, Builder, Strategy	Notification factory, discount strategy
4	Adapter, Decorator, Facade, Observer + UML	Payment adapter, order events in NestJS
5	Easy problems	Parking Lot, Library, Tic-Tac-Toe
6	Medium problems	Vending Machine, Splitwise, ATM

Where to learn more

- **refactoring.guru/design-patterns**: the best free site, with pictures and TypeScript code
- **Head First Design Patterns** (book): very easy English

- GitHub **"awesome-low-level-design"**: problems with solutions

24. Summary checklist

- [ ] **OOP:** class, object, encapsulation, abstraction, inheritance, polymorphism
- [ ] **Relationships:** IS-A vs HAS-A (composition, aggregation), association
- [ ] **SOLID:** one job, extend don't edit, child = parent, small interfaces, depend on interfaces
- [ ] **UML:** draw class boxes with arrows
- [ ] **8 patterns:** Singleton, Factory, Builder, Adapter, Decorator, Facade, Strategy, Observer
- [ ] **Principles:** DRY, KISS, YAGNI, composition over inheritance
- [ ] **Interview steps:** clarify, then entities, relations, methods, patterns, code, extend
- [ ] **Practise:** Parking Lot, Library, Vending Machine, Splitwise, BookMyShow

> You already use most of these every day in NestJS. LLD gives them **names** and teaches you to **explain** them.